Technical report
(draft, October 2026)

Nathan W. Brixius · Jean-Pierre Goux · Claude · and friends[1]

A certified doubly nonnegative bound for branch and bound on the quadratic assignment problem, and nug30 revisited

October 29, 2026

Abstract. We describe a certified lower bound for the quadratic assignment problem (QAP) based on the doubly nonnegative (DNN) semidefinite relaxation, and its use in the branch-and-bound algorithm that solved the nug30 problem in 2000. The bound is the dual value of an ADMM multiplier whose projection onto the relaxation's face is negative semidefinite by construction, evaluated by a Gilmore–Lawler computation on the lifted cost matrix, less explicit floating-point allowances, so that it is a valid bound after any number of iterations. In the branch-and-bound algorithm the new bound is computed only near the top of the tree, at nodes that the quadratic programming bound QPB does not fathom, and is used only to fathom; QPB continues to supply branching information and to eliminate children. Together with a new branching rule based on the geometry of grid instances, this solves nug30 in 5,817 nodes and under two hours on a four-core cloud computer, and in 20 minutes on a laptop, a problem whose solution in 2000 required 1.19×10^{10} nodes and a week on a computational grid of hundreds of workstations.

Key words. Quadratic assignment problem – branch and bound – semidefinite programming – doubly nonnegative relaxation

---

1. Introduction

The quadratic assignment problem (QAP) is a standard problem in location theory. The QAP in "Koopmans–Beckmann" form is to

$$\min_{\pi} \sum_{i=1}^{n} \sum_{k=1}^{n} a_{ik}\, b_{\pi(i),\pi(k)} + \sum_{i=1}^{n} c_{i,\pi(i)},$$

where a_{ik} is the flow between facilities i and k, b_{jl} is the distance between locations j and l, c_{ij} is the fixed cost of assigning facility i to location j, and $\pi(i) = j$ if facility i is assigned to location j. The problem can alternatively be represented in matrix form as

$$\begin{aligned} \mathrm{QAP}(A,B,C): \quad & \min \ \mathrm{tr}(AXB+C)X^T \\ & \text{s.t. } X \in \Pi, \end{aligned} \tag{1}$$

[1] Claude is an AI system made by Anthropic. Source code, data and run logs: the `cognac` branch of `https://github.com/natebrix/qap`. See the acknowledgments for who did what.

where Π is the set of $n \times n$ permutation matrices and $X_{ij} = 1$ if facility i is assigned to location j. Throughout we assume that at least one of A and B is symmetric. Replacing the other by its symmetric part then changes no objective value, so we may, and do, take both to be symmetric. QAPLIB [6] has long served as the benchmark library for the QAP; for surveys see [5, 12].

The QAP is NP-hard, and in practice has proven to be one of the hardest discrete optimization problems to solve to optimality. Exact methods are almost exclusively branch and bound, and their success rests on the lower bound. For implementation in a branch-and-bound algorithm the relationship between the quality of a bound and its computational cost is extremely important, as is the quality of the branching information that bound computations provide. The Gilmore–Lawler bound (GLB) [8, 11] is cheap but weak for larger problems; bounds based on linear programming and its duals, on eigenvalues [10], and on semidefinite programming (SDP) [16] are progressively stronger and more expensive. A convex quadratic programming bound QPB [1] occupies a useful middle ground: better than GLB and the projected eigenvalue bound PB [10], at far lower cost than LP or SDP bounds, and it provides a dual matrix that serves both to eliminate children and to choose how to branch [3].

The most commonly used benchmark QAPs are the "Nugent" problems, introduced in 1968 [13], whose distance matrices are Manhattan distances on rectangular grids. Their solution has marked advances in both processor capability and QAP solution methods: nug20 was first solved in 1995, nug21 and nug22 in 1997, and nug25 in 1998, each on a parallel computer with tens to hundreds of processors (see [2] for the history). In June 2000 Anstreicher, Brixius, Goux and Linderoth [2] solved nug30 with a QPB-based branch and bound running on a computational grid: an average of 653 worker machines over 6.9 days, the equivalent of almost 7 years of computation on a single HP9000 C3000 workstation, in a tree of 1.19×10^{10} nodes. The same code then solved kra30b, kra32 and tho30. Since then, semidefinite relaxations have become practical inside branch and bound: Fujii et al. [7] bounded every node of a branch-and-bound tree with a Lagrangian-DNN relaxation and, on large parallel computers, solved nug30-sized instances with tens of thousands of nodes as well as several QAPLIB instances that had remained open.

In this paper we revisit nug30 with the branch-and-bound algorithm of [3, 2], on a single computer. Three changes make this possible:

(C1) a *certified* DNN bound (Section 2): the doubly nonnegative relaxation of Zhao, Karisch, Rendl and Wolkowicz [16], in the facially reduced form of Oliveira, Wolkowicz and Xu [14] and solved by their ADMM, with a lower bound that is valid at every iteration. It is within 3% of the optimum at the root of nug30, against 13% for QPB, but each evaluation takes seconds rather than milliseconds;

(C2) a new branching rule (Section 3.2) that branches on the *most eccentric free location*, the free location farthest from the others, at no cost per node, and a cheaper lookahead rule built on it;

(C3) a division of labor (Section 3.3): the DNN bound is computed only at the top of the tree, at nodes that QPB does not fathom, and is used only to fathom. QPB continues to supply the bound used for branching and the reduced costs used to eliminate children, and does all the work below.

With these, nug30 is solved in 5,817 nodes and 1.7 hours on four cores of a cloud computer, and in 20 minutes on a laptop with eight solver processes (Section 4). In Section 2 we describe the DNN relaxation, the certificate and its implementation. In Section 3 we review the branch-and-bound algorithm of [2] and describe the new branching rules and the use of the DNN bound. In Section 4 we give bounds for the QAPLIB problems of [1], the effect of the DNN bound on the Nugent problems, and the solution of nug30. The validation of the bound is described in Appendix A.

Notation. We use $\mathrm{tr}(\cdot)$ to denote the trace of a matrix. If A and B are matrices then $\langle A,B\rangle = \mathrm{tr}(AB^T)$, $A\otimes B$ is the Kronecker product of A and B, and $\mathrm{vec}(A)$ is the vector formed by "stacking" the columns of A. We use e to denote a vector with each component equal to one, $\mathscr{S}^N$ for the symmetric $N\times N$ matrices, and $\lambda_{\max}(\cdot)$ for the largest eigenvalue of a symmetric matrix. For convenience we let the name of an optimization problem, like $\mathrm{QAP}(A,B,C)$, also refer to its optimal value.

2. A certified DNN bound

2.1. The DNN relaxation

Consider a node of the branch-and-bound tree with m facilities left to assign. Its subproblem is again a QAP (1) of size m, with reduced matrices $A,B\in\Re^{m\times m}$, a linear term C that collects the interactions with the fixed assignments, and a constant s. For $X\in\Pi$ let $x=\mathrm{vec}(X^T)$, ordered so that x_{ij} (facility i at location j) has lifted index $1+im+j$, and let $y=(1,x)$ and $N=m^2+1$. Define the symmetric lifted cost $L\in\mathscr{S}^N$ by

$$L_{(ij),(kl)} = \tfrac{1}{2}(a_{ik}b_{jl}+a_{ki}b_{lj}), \qquad L_{0,(ij)} = L_{(ij),0} = \tfrac{1}{2}c_{ij}, \qquad L_{00}=0. \tag{2}$$

Then for every permutation, $\langle L, yy^T\rangle = \sum a_{ik}b_{jl}x_{ij}x_{kl} + \sum c_{ij}x_{ij}$, the subproblem objective without s, for any A and B, symmetric or not.

The face. The constraints $Xe=X^Te=e$ say that y lies in the null space of a linear map, so every lifted permutation $Y=yy^T$ has its range in a fixed subspace of dimension $(m-1)^2+1$. Let $V\in\Re^{N\times K}$, $K=(m-1)^2+1$, be an orthonormal basis of it. Zhao et al. [16] take $V=\begin{pmatrix}1 & 0\\ e/m & V_m\otimes V_m\end{pmatrix}$, suitably normalized, where the columns of $V_m\in\Re^{m\times(m-1)}$ span $e^\perp$. Every lifted permutation then satisfies $Y=VRV^T$ with $R\succeq 0$. The DNN relaxation of the subproblem is

$$\begin{aligned}\mathrm{DNN}(A,B,C) = s + \min\quad & \langle L,Y\rangle \\ \text{s.t.}\quad & Y=VRV^T,\ R\succeq 0,\ Y_{00}=1,\ \mathscr{G}(Y)=0,\ 0\le Y\le 1,\end{aligned} \tag{3}$$

where $\mathscr{G}$ selects the "gangster" entries $Y_{(ij),(il)}$, $l\neq j$, and $Y_{(ij),(kj)}$, $k\neq i$, which vanish for permutations since a facility has one location and a location one facility. Oliveira, Wolkowicz and Xu [14] solve (3) by ADMM on the splitting $Y=VRV^T$, alternating

$$R \leftarrow \mathscr{P}_+\big(V^T(Y+Z/\beta)V\big), \tag{4}$$

$$Y \leftarrow \Pi_{\mathscr{Y}}\big(VRV^T-(L+Z)/\beta\big), \tag{5}$$

$$Z \leftarrow Z+\gamma\beta\,(Y-VRV^T), \tag{6}$$

where $\mathscr{P}_+$ projects onto the positive semidefinite cone, $\mathscr{Y} = \{Y : Y_{00} = 1, \mathscr{G}(Y) = 0, 0 \le Y \le 1\}$ is a box with fixed entries (so $\Pi_{\mathscr{Y}}$ is entrywise clipping), $\beta > 0$ is the penalty and $\gamma \in (0, \frac{1+\sqrt{5}}{2})$ the dual step.

2.2. A bound from any multiplier

ADMM iterates are neither primal feasible nor dual optimal, and the primal value $\langle L, Y\rangle$ is not a bound: stopped early, it can lie above the optimum. Branch and bound needs a number that is provably below the subproblem optimum after any number of iterations. We obtain one from the multiplier.

Lemma 1. *Every lifted permutation $Y = yy^T$ satisfies $Y = VRV^T$ with $R = V^TYV \succeq 0$ and* $\operatorname{tr}(R) = m+1$.

Proof. Since y lies in the range of V,

$$Y = VV^TYVV^T = VRV^T, \qquad R = V^TYV = (V^Ty)(V^Ty)^T \succeq 0,$$

and $\operatorname{tr} R = \|V^Ty\|^2 = \|y\|^2 = 1 + \sum x_{ij}^2 = m+1$. □

Proposition 2. *Let $Z \in \mathscr{S}^N$ be any symmetric matrix, $\lambda \ge \lambda_{\max}(V^TZV)$, and let $\mathscr{P} \supseteq \{yy^T : X \in \Pi\}$. Then the subproblem optimum is at least*

$$s + \min_{Y \in \mathscr{P}} \langle L+Z, Y\rangle - (m+1)\lambda. \tag{7}$$

Proof. For a lifted permutation $Y = VRV^T$ (Lemma 1), since $R \succeq 0$,

$$\langle Z, Y\rangle = \langle V^TZV, R\rangle \le \lambda_{\max}(V^TZV)\operatorname{tr} R \le (m+1)\lambda.$$

Hence

$$\begin{aligned}\langle L, Y\rangle = \langle L+Z, Y\rangle - \langle Z, Y\rangle &\ge \langle L+Z, Y\rangle - (m+1)\lambda \\ &\ge \min_{\mathscr{P}} \langle L+Z, \cdot\rangle - (m+1)\lambda.\end{aligned}$$ □

The same bound appears as the dual function of Graham et al. [9, eq. (3.11), Thm. 3.4]. Proposition 2 holds for every Z; the quality of the bound depends on Z being near a dual optimum of (3), and on choosing Z so that λ can be small. Two choices of $\mathscr{P}$ and one of Z make it both cheap and tight.

The multiplier. The prototype of [14] projects V^TZV onto the negative semidefinite cone, which costs an extra eigendecomposition per check. Instead we use the multiplier implied by the R-step. Let $W = V^T(Y + Z/\beta)V$ be the matrix projected in (4), and

$$Z' = Z + \beta(Y - VRV^T), \qquad \text{so that} \qquad V^TZ'V = \beta(W - \mathscr{P}_+(W)) \preceq 0.$$

The last matrix is β times the negative part of W, negative semidefinite by construction. This holds even when the eigenvectors used for $\mathscr{P}_+$ are inaccurate: if $\mathscr{P}_+(W)$ is formed as W minus the computed negative part $\sum_{w_k<0} w_k v_k v_k^T$, the difference is exactly that sum. In floating point, $\lambda_{\max}(V^TZ'V)$ is a tiny positive number, which we bound below.

The polyhedral part. With $\mathscr{P}$ the box $\mathscr{Y}$ itself, $\min_{\mathscr{P}}\langle M,Y\rangle$ for $M = L + Z'$ is M_{00} plus the sum of the negative entries of M outside the gangster set: the prototype's choice. A much tighter, still cheap choice uses that every permutation is an assignment in two levels. Write $M_{(ij),\cdot}$ for row (ij) of M. For a permutation σ,

$$\langle M, yy^T\rangle = M_{00} + \sum_i \Big(2M_{0,(i\sigma_i)} + M_{(i\sigma_i),(i\sigma_i)} + \sum_{k\neq i} M_{(i\sigma_i),(k\sigma_k)}\Big). \tag{8}$$

The inner sum is a perfect matching between facilities $k \neq i$ and locations $l \neq \sigma_i$, so it is at least the value LAP_{ij} of the $(m-1)\times(m-1)$ assignment problem on the block $M_{(ij),(kl)}$, $k \neq i$, $l \neq j$. Hence

$$\min_\sigma \langle M, yy^T\rangle \;\geq\; \mathrm{GL}(M) := M_{00} + \mathrm{LAP}\big(c\big), \qquad c_{ij} = 2M_{0,(ij)} + M_{(ij),(ij)} + \mathrm{LAP}_{ij}, \tag{9}$$

a Gilmore–Lawler computation [8, 11] on the lifted matrix. It also bounds the DNN relaxation itself, not only permutations: for feasible Y of (3), column (ij) restricted to $k \neq i$, $l \neq j$ is nonnegative with all row and column sums $x_{ij} = Y_{0,(ij)}$, so it is x_{ij} times a doubly stochastic matrix, and assignment duality applies twice. It is never below the box value and reaches a given value 1.5–6× sooner in the nug20 tests. Any lower bound on each LAP_{ij} may replace it, for example the row-then-column reduction, which costs $O(m^2)$. The implementation uses the reduction, plus exact Hungarian solves when the result is within a small gap of the cutoff.

Remark 3. Nothing in the certificate depends on the ADMM having converged, on the accuracy of its eigenvectors, or on the single-precision arithmetic used in the R-step (Section 2.4). Only the multiplier Z', the Cholesky test below and (9) enter the certified value, all in double precision.

2.3. Floating-point safety

We assume IEEE binary64 arithmetic with rounding to nearest, an extended type (`long double`) with a p-bit significand ($p = 64$ on x86), and no overflow or underflow; a non-finite value anywhere yields no certificate. The certified value is $s + \mathrm{GL}(L+Z') - \mu$, rounded down, where the inputs and the margin μ cover:

(o) the subproblem data: s and the reduced linear term C, sums of products that can cancel, are replaced by lower bounds (exact when all operands are multiples of $\frac{1}{2}$ and the sum of magnitudes is below 2^{p-2}, as for integer data; otherwise the extended-precision sum less $(k+2)\,2^{-p}\sum|t|$ for k terms t, rounded down). Bounds for the lowered subproblem are bounds for the exact one, since assignment variables are nonnegative;

(i) $(m+1)\lambda$ with $\lambda \geq \lambda_{\max}(V^T Z' V)$, obtained by Rump's criterion [15]: if the floating-point Cholesky factorization of $\delta I - S$ succeeds for the computed $S \approx V^T Z' V$, then $\lambda_{\max}(S) \leq \delta + c$ with an explicit c depending on the dimension, the unit roundoff u and $\mathrm{tr}(\delta I - S)$. To this is added $64\,m\,u\,\|Z'\|_F$ for the error of computing S through Householder transforms rather than with an exact V;

(ii) the summation and rounding errors of (9), accumulated in extended precision: $8(m+2)u\,\|M\|_1$, plus $(m+2)u\sum_{(ij),(kl)}\frac{1}{2}(|a_{ik}b_{jl}|+|a_{ki}b_{lj}|)$ for forming and scaling L.

These are standard error bounds with generous constants, not interval arithmetic, and not every constant is derived here: the allowance $64\,m\,u\|Z'\|_F$ for the Householder transforms rests on a standard backward-error argument and was checked empirically (errors at most $2.1\,u\|Z'\|_F$), not proved. At $m = 28$ the largest margin seen was about 10^{-7} in objective units, against integer objective values.

2.4. Implementation

The bound is implemented in C++ (`latest/dnn.C`, about 1200 lines) with LAPACK and BLAS (OpenBLAS). A numpy prototype (`experiments/dnn_bound`) was ported faithfully; a prototype mode reproduces its certified values to 10^{-7}.

Products with V. The prototype forms V explicitly and computes V^TMV and VRV^T with dense products of order $N \times N$, which costs $O(m^6)$. We take for V_m the first $m-1$ columns of the Householder reflector H with $He_{m-1} = e/\sqrt{m}$. Then V^TMV and VRV^T are a reflection along each of the four tensor modes plus a 2×2 rotation coupling the constant coordinate, $O(m^4)$ per product. Both the R-step and the certificate are invariant under an orthogonal change of basis of the face, so any orthonormal V gives the same iterates and bound.

Eigensolver. Each R-step needs the projection of a $K \times K$ symmetric matrix ($K = 730$ at $m = 28$) with hundreds of eigenvalues of each sign, so a partial spectrum does not help directly. Between iterations the matrix changes little. We track its negative eigenspace by a Chebyshev-filtered subspace iteration in single precision (BLAS level 3), with a full single-precision eigendecomposition every 20 iterations. Because R is formed as W minus the computed negative part, the certificate is unaffected. When a check fails the Cholesky test, or the semidefinite penalty alone decides whether the node is fathomed, the multiplier is re-projected in double precision before certifying.

Parameters. The objective is scaled to $\max|L| = 1$. We use $\beta = 0.3 \cdot m/3$ (the prototype's $m/3$ times 0.3), which certifies 2–3$\times$ sooner on hard nodes, and $\gamma = 1.618$. A certificate is computed every 10 iterations, and the run stops at the first certified value above the cutoff. An optional give-up rule stops a node early when its certified value, extrapolated generously from its recent gains, stays below the cutoff. This only affects cost: the node is then branched on as usual.

Warm starts. When the DNN bound at a node misses, its children can start their ADMM from its final multiplier, with the row and column of the new assignment removed from the lifted indices. Only Z is carried over; starting Y from the parent made children slower. This cuts the iterations to fathom a child by about 40%, but less of the time, because a warm-started ADMM pays for many negative eigenvalues from its first iteration where a cold one starts cheap.

Table 1. The certified DNN bound in C++ against the numpy prototype, CPU seconds per node, cutoff = optimum -1.95 (one core; prototype times from the studies in `experiments/dnn_bound`).

node set	fathomed (both)	C++	prototype	ratio
nug20, depth 2 (all 98, $m = 18$)	96	0.62	8.35	13×
nug30, depth 2 (36 random, $m = 28$)	34	9.6	139	14.5×
nug30, depth 2 (8 lowest QPB)	0	11 (200 it.)	397	36×
nug30, depth 3 (20, $m = 27$)	19	7.4	121	16×

Speed. Table 1 compares the C++ bound with the prototype on the same nodes. Measured back to back on a loaded 4-core Xeon, an ADMM iteration at $m = 28$ costs 43 ms against 339 ms. Per node, the C++ bound needs fewer iterations to certify, thanks to the tighter polyhedral part and the smaller β, and is 13–36× faster.

3. The branch-and-bound algorithm

In this section we review the branch-and-bound algorithm of [3, 2], describe the changes made to it since 2000, including a new branching rule, and describe how the DNN bound is used.

3.1. QPB, elimination and branching

The quadratic programming bound of [1] is

$$\mathrm{QPB}(A,B,C): \quad \min\ \mathrm{vec}(X)^T Q\,\mathrm{vec}(X) + \langle C, X\rangle + \gamma \quad \text{s.t.} \quad Xe = X^T e = e,\ X \geq 0,$$

where $Q = (B \otimes A) - (I \otimes S) - (T \otimes I)$, and S, T and γ come from the spectral decompositions of $V^T AV$ and $V^T BV$; Q is positive semidefinite on the null space of the equality constraints, and $\mathrm{PB}(A,B,C) \leq \mathrm{QPB}(A,B,C) \leq \mathrm{QAP}(A,B,C)$. As in [3, 2] we compute it approximately by the Frank–Wolfe algorithm, which is known to have poor asymptotic performance, but in the context of QPB is attractive because each iteration is dominated by a single matrix multiplication and the solution of a dense linear assignment problem. It produces a lower bound z and a dual matrix $U \geq 0$ such that $\mathrm{tr}(AXB + C)X^T \geq z + \langle U, X\rangle$ for every permutation matrix X. If v is the incumbent value, then $z + U_{ij} > v$ implies that $X_{ij} = 0$ in any improving solution.

Branching is "polytomic": child problems are created either by assigning a fixed facility to all available locations (*row branching*) or by assigning all available facilities to a fixed location (*column branching*), and logic based on U eliminates children in both cases. Where the symmetry of a grid instance can be exploited, only row branching on restricted location sets is used. The rules of [2] used here are:

> **Rule 2.** Branch on the row or column that produces the smallest number of children, breaking ties by the largest sum of the remaining U_{ij}.
>
> **Rule 4.** For the candidate rows and columns with the largest U sums, bound every child by QPB and branch on the line whose children score best, using their

bounds and their own dual matrices. Rule 4 is a *strong branching*, or *look-ahead*, rule.

A rule is chosen at each node from a table of strategies by the node's depth and its relative gap $g = (v - z')/(v - z_0)$, where z' is the bound inherited from the parent and z_0 the root bound: the strategies are scanned from the most expensive to the least until one is found whose gap tolerance is below g and whose depth limit is at least the node's depth.

Changes since 2000. Several changes improve QPB's efficiency without changing what it computes. Each node's Frank–Wolfe iterations start from its parent's solution, with the fixed row and column removed, instead of from the uniform matrix, and the bound kept is the best (bound, U) pair over all iterations rather than the last; on nug24 this alone saves 13% of the time. A bug in the code of 2000 made Rule 4 rank its candidate lines essentially arbitrarily; corrected, it saves 4% on our standard test set. Rule 4's child bounds now use fewer Frank–Wolfe iterations (NFW3 of 25 at depths 1–2 and 12 below), starting from the parent's solution; this saves 24% on the standard set. On nug25, with the optimum as the incumbent, the resulting code takes 52,989,998 nodes and 18,353 CPU seconds, against 7.1×10^7 nodes and 1.1×10^6 CPU seconds on the processors of 2000 [2, Table 3].

3.2. Branching on eccentric locations

On grid instances, locations differ in a way the dual matrix U does not see. A facility at a corner or an edge of the grid is far from many others, so fixing a facility there creates children with large, spread-out linear terms, and more of them fathom. We define the *eccentricity* of a free location j as the largest distance from j to another free location, $\max\{b_{jl} : l \neq j \text{ free}\}$.

Rule 6. Branch on the column (location) of largest eccentricity, breaking ties by the number of children eliminated and then by the U sum, as in Rule 2. If all free locations are equally eccentric, use Rule 2.

Rule 7. Rule 4 restricted to a smaller candidate set: the NBEST most eccentric columns and the NBEST-1 rows with the largest U sums.

Rule 6 costs nothing per node, unlike lookahead, which bounds every child of every candidate line. Used instead of lookahead at depths 3–4, it makes dense instances much faster (nug20 2.1×, nug21 1.7×, tai17a 2.0×, 1.3× on our standard test set), but sparse ones, where lookahead pays for itself, slower (chr15a 1.4× slower, esc16a more than 2.9× slower; esc16a's hypercube distances make every location equally eccentric). Rule 7 keeps lookahead where it pays and bounds fewer lines: against Rule 4 with five rows and five columns, it takes 0.80 of the time on nug20, 0.87 on nug21, 0.79 on tai17a and 0.69 on esc16a, and 1.13 on chr15a, a geometric mean of 0.84. The rows keep it working where locations do not differ. Table 2 gives the strategy used for the grid instances in this paper, with Rule 7 at depths 3–4 and Rule 6 down to depth 6. The 2000 solve of nug30 used Rule 4 down to depth 7 and Rule 2 below, with gap thresholds from 0.34 to 0.03 [4, Table 5.11]. On nug25, with QPB alone, it takes 40,183,792 nodes

Table 2. The branching strategy for grid instances (`PARAM/nug_deep.par`). NFW1: Frank–Wolfe iterations for the bound; NFW2: when the node cannot be fathomed; NFW3: for the bounds of lookahead children; NBEST: candidate lines; UPDATE: iterations between updates of S, T.

Gap	Depth	Rule	NFW1	NFW2	NFW3	NBEST	UPDATE
.20	2	4	150	100	25	30	30
.12	4	7	150	100	12	3	30
.05	6	6	100	50	–	–	30
0	–	2	100	50	–	–	30

and 12,362 CPU seconds, against 52,989,998 nodes and 18,353 seconds for the default strategy (Rule 7 at depths 3–4 and Rule 2 below).

3.3. Using the DNN bound to fathom

The DNN bound is added to the algorithm as follows.

1. At a node of depth $d \in [d_1, d_2]$ whose QPB value does not fathom it, and whose relative gap is at most a given $g_{\max}$, the certified DNN bound is computed for its reduced subproblem (`bnbdrv -x d1:d2[:gap[:giveup[:warm]]]`).

2. If it exceeds the incumbent minus the fathoming tolerance, the node is fathomed. Otherwise it raises the node's recorded bound, which its children inherit and which also feeds the choice of strategy and some branching scores.

3. Branching, and the elimination of children by $z + U_{ij} > v$, use the QPB value and its dual matrix U, never the DNN value. The elimination argument is a property of the Frank–Wolfe linearization of QPB, and holds only with the bound it is paired with.

Near the top of the tree nodes are few, and each carries a subtree of 10^3 to 10^9 QPB nodes, so a bound that costs seconds pays for itself many times over when it fathoms. Deep in the tree it would not. Combining bounds is not new in itself: earlier experiments that mixed QPB with GLB [4] switched the bound wholesale, using one bound for both bounding and branching at a node, and did not pay off, both because GLB is weak even deep in the tree and because each bound was then doing the job it is worse at. Here each bound is pointed at the job it does well. Nor does the DNN bound provide useful branching information cheaply: the outer assignment problem of (9) yields, with its dual potentials u and v, a valid bound $\mathrm{GL}(M) + r_{ij}$ for every child $x_{ij} = 1$ of a node the bound misses, with $r_{ij} = c_{ij} - u_i - v_j$, at no extra cost. On nine sampled depth-1 and depth-2 misses of nug30 it fathomed none of their 784–841 children each: near a dual optimum the outer costs are almost flat (median r_{ij} 0.1, largest 12.1, where the nearest miss needed 14.4), and even forcing the assignment in the outer problem gains at most 12.1. A child's gain comes from re-solving the relaxation with the extra fixing.

Where to apply it. Two studies with the prototype measured the bound on QPB-open nodes (Table 3). At depth 1 it rarely fathoms. At depth 2 it fathoms nearly all nodes of nug20, and 34 of a random sample of 36 of nug30's 225 (94%). In that sample the outcome was almost a function of the node's QPB value: every node with QPB below about 5548 was missed, every node above about 5554 fathomed. Applied to the whole depth-2 frontier, that threshold predicts about 185 of 225 fathoms (82%), a sample-based estimate rather than a guarantee for other depths or instances. At depth 3, under the depth-2 nodes it misses, it fathoms almost everything. The misses are the most expensive subtrees, so the bound has to be applied at several consecutive depths. In the full solves, depths 2 to 6 or 7 worked best.

The two bounds share a lineage: QPB is built on the projected eigenvalue bound PB [10], and the DNN relaxation on the same projection onto the assignment constraints. To see whether that is why QPB predicts the DNN outcome, we computed GLB, PB and QPB at all 225 depth-2 nodes of nug30 and ran the DNN bound on each (165 are fathomed within 1,000 iterations, the same count as in the full solves). Measured by the area under the ROC curve for separating fathomed from missed nodes, QPB is the best predictor (0.990, and 0.994 for its parametric form), the lookahead estimate the solver already has before it bounds the node is close behind (0.988), then GLB (0.969) and PB (0.920). At its best threshold QPB classifies 96.9% of the nodes correctly, GLB 93.8% and PB 86.2%; with QPB, fathomed and missed nodes overlap only in a band of about 25 units around 5590. So the purely spectral PB, from which QPB descends, is the weakest predictor, and the purely combinatorial GLB, a far weaker bound, is a good one. QPB combines a spectral part with assignment problems, much as the DNN certificate combines a semidefinite part with Gilmore–Lawler assignment problems, and it is that combination, more than the spectral lineage alone, that makes the cheap bound a good guide to the expensive one. (One instance at one depth: suggestive rather than conclusive.)

Does it follow that branching choices made with QPB also serve the DNN bound? We took five depth-2 nodes the DNN bound misses and, for nine candidate lines at each (four rows, four columns and the most eccentric free location), bounded all 1,260 children by QPB and by the DNN bound with the solver's settings. At the two near-miss nodes every child of every line is fathomed, so the choice does not matter. At the farthest miss the lines give 6 to 17 child misses, and QPB ranks them well: the rank correlation between misses and the average child QPB is 0.92, and 0.98 for the number of children with QPB above the level's threshold. In between the ranking is noisy, and the average child QPB, Rule 4's score, sometimes orders the lines backwards. The most eccentric location's line was as good as the best line at four of the five nodes and one child miss worse at the fifth. So where the choice matters QPB-based branching picks good lines for the DNN bound, and Rule 6 does as well at no cost; but most DNN misses are near misses, where it does not matter, which is why more lookahead at these depths changes little.

Fathoming tolerance. The fathoming tolerance is 1.95 when all objective values are provably even, as for the nug instances, 0.95 when they are integers, and 0 otherwise; data is treated as integer only if every objective value is below 2^{53}, so that it is computed exactly. For even values and an even incumbent, an improving solution is at most the

Table 3. Root bounds and DNN fathoming at the top of the tree (prototype studies; QPB from the solver; incumbent = optimum).

	nug12	nug20	nug30
optimum	578	2570	6124
root QPB	478.5	2243.9	5332.3
root DNN (certified)	568.0	2506.3	5949.3
depth 1: fathomed / QPB-open	–	1 / 6	0 / 1 sampled
depth 2: fathomed / QPB-open	–	96 / 98	34 / 36 sampled
depth 3: fathomed / QPB-open	–	200 / 200 sampled	19 / 20 sampled

incumbent minus 2, so 1.95 leaves a cushion of 0.05 for rounding.

Scope. The DNN certificate covers only the DNN prunes. QPB values, QPB reduced-cost eliminations and symmetry reductions are computed in floating point without such margins, as in [2], so the search as a whole is not an independently checkable proof. The full solver also needs one of A, B to be symmetric (otherwise QPB is not valid, and the driver warns), and its QPB search is currently wrong on data with negative entries in A or B (Appendix A). The DNN bound has neither restriction.

Parallel implementation. The 2000 solve used the master-worker library MW on the Condor system to distribute subtrees to hundreds of workers. Here a small driver (`bench/parallel.py`) does the same on one machine: the root is solved serially, its children go into a pool, and one solver process per core each take a chunk of nodes, returning their unfinished nodes when idle workers need work. Search statistics, including DNN counts, are kept in checkpoint files, so a run can be stopped, resumed and split across processes with identical totals. The DNN bound's floating-point results depend on the BLAS kernels selected for the CPU, so node counts can differ between machines if a certified value lands within about 10^{-6} of a cutoff. Validity is not affected.

4. Computational results

All runs use the known optimum as the incumbent, so they measure the work of the proof; the runs of [2] started from the best known value plus 2 and so also had to recover an optimal solution. They ran on cloud virtual machines with 4 cores (Intel Xeon, 2.1–2.8 GHz), with four solver processes, and on an Apple M2 Pro laptop (6 performance and 4 efficiency cores), with eight solver processes and Apple's Accelerate library for BLAS and LAPACK. CPU is the sum over processes.

4.1. Bounds for QAPLIB problems

In Table 4 we give, for the problems of [1, Table 1], the optimum value and the lower bounds GLB, PB, QPB and SDPB1 reported there, together with the certified DNN bound at the root. GLB, PB and SDPB1 (the cheapest SDP relaxation of [16]) are taken from [16], and QPB is the bound computed in [1] with one step of its dual improvement

(QPB1). The DNN bound was computed with up to 4,000 ADMM iterations and no cutoff. All bounds are rounded up to the next integer, since the data for these problems is integral.

From Table 4 it is clear that the DNN bound is much stronger than QPB and SDPB1, on every problem but kra30a and kra30b (below). Over the remaining 40 problems it closes on average 89% of the gap between QPB and the optimum, and at least 62% on every problem. It equals the optimum on ten of them: esc16j, all of had12–had20, rou12, scr12, scr15 and tai12a. On the scr problems, where the LP-based bounds are much better than QPB [1], it is at most 3% below the optimum, and on the Nugent problems on average 1.8%. It is also far more expensive: from about a second for the esc16 problems to five CPU minutes for tai30a and tho30, and most bounds with $n \geq 14$ were still rising when the iteration limit was reached, so the values in the table are lower bounds on the value of the DNN relaxation. For kra30a and kra30b the ADMM did not converge with the default parameters, which were tuned on the Nugent problems: the certified values, about 17,400 and 17,600, are valid but far below QPB, and we omit them pending a study of the parameters for these problems.

4.2. The DNN bound in the search

Table 5 summarizes solves of the Nugent problems with and without the DNN bound. The two nug25 rows with `nug_deep` and the 2–5 DNN setting isolate the DNN bound: the same strategy, rules and symmetry handling, with the bound on or off. Other comparisons in the table change the strategy as well. At nug20 the DNN bound at depths 2–3 removes all but 141 of 526,205 nodes and halves the time. At nug24 the tree collapses to 299 nodes. From nug25 on, the DNN bound does most of the work: on nug28, 12,926 of 12,976 CPU seconds are spent in it, on 1,727 nodes of which 1,664 are fathomed.

4.3. The solution of nug30

nug30 was solved three times. The first solve applied the DNN bound at depths 2–5. QPB search below the depth-5 nodes the bound missed grew to about 12,000 CPU seconds, so for the remaining work (506 open nodes, including a whole depth-1 subtree) the bound was extended to depth 6, which finished in 21 minutes. In total, 45,486,332 nodes and 40,555 CPU seconds; the DNN bound ran on 5,615 nodes, fathomed 5,403, and accounted for 26,770 of those seconds. A run with the DNN bound at depths 2–6 throughout took 6,532,419 nodes, 18,614 CPU seconds and $\approx$1.3 h of wall time: the DNN bound ran on 5,764 nodes, fathomed 5,551, and accounted for 17,192 CPU seconds, 92% of the total. Almost all of the difference from the first solve is QPB search below depth-6 nodes that the depth-5 setting left to QPB.

Two changes target that run's remaining costs. The QPB search below its 72 depth-7 nodes took about 1,300 CPU seconds; applying the DNN bound at depth 7 too fathoms all of them. And most DNN calls are at children of nodes whose own DNN bound missed, which can be warm-started. The run with both (`-x 2:7:1:300:1`) ran on a different machine, so the depth-6 settings were rerun there with the same code (the last two rows of Table 5). The rerun reproduces the earlier tree exactly, and the new settings

Table 4. Bounds for QAPLIB problems. GLB, PB, QPB and SDPB1 from [1, Table 1]; DNN: the certified bound of Section 2 at the root, after at most 4,000 ADMM iterations; –: held out (see text).

Problem	OPT	GLB	PB	QPB	SDPB1	DNN
esc16a	68	38	47	55	47	64
esc16b	292	220	250	250	250	290
esc16c	160	83	95	95	95	154
esc16d	16	3	−19	−19	−19	13
esc16e	28	12	6	6	6	27
esc16g	26	12	9	9	9	25
esc16h	996	625	708	708	708	977
esc16i	14	0	−25	−25	−25	12
esc16j	8	1	−6	−6	−6	8
had12	1652	1536	1573	1592	1604	1652
had14	2724	2492	2609	2630	2651	2724
had16	3720	3358	3560	3595	3612	3720
had18	5358	4776	5104	5143	5174	5358
had20	6922	6166	6625	6677	6713	6922
kra30a	88900	68360	63717	68572	69736	–
kra30b	91420	69065	63818	69021	70324	–
nug12	578	493	472	482	486	568
nug14	1014	852	871	891	903	1011
nug15	1150	963	973	996	1009	1141
nug16a	1610	1314	1403	1448	1461	1600
nug16b	1240	1022	1046	1071	1082	1219
nug17	1732	1388	1487	1529	1548	1708
nug18	1930	1554	1663	1705	1723	1894
nug20	2570	2057	2196	2254	2281	2507
nug21	2438	1833	1979	2055	2090	2382
nug22	3596	2483	2966	3080	3140	3529
nug24	3488	2676	2960	3028	3068	3402
nug25	3744	2869	3190	3272	3305	3626
nug30	6124	4539	5266	5365	5413	5950
rou12	235528	202272	200024	206102	208685	235528
rou15	354210	298548	296705	303777	306833	350217
rou20	725520	599948	597045	607822	615549	695181
scr12	31410	27858	4727	8585	11117	31410
scr15	51140	44737	10355	12479	17046	51140
scr20	110030	86766	16113	23960	28535	106802
tai12a	224416	195918	193124	199597	203595	224416
tai15a	388214	327501	325019	330310	333437	377101
tai17a	491812	412722	408910	416033	419619	476525
tai20a	703482	580674	575831	585139	591994	671675
tai25a	1167256	962417	956657	983456	974004	1096657
tai30a	1818146	1504688	1500407	1518059	1529135	1706870
tho30	149936	90578	119254	124684	125972	143576

Table 5. Branch and bound with and without the DNN bound (4 workers on a cloud computer unless noted). QPB only: default strategy unless noted. DNN: `-x d1:d2`, giving up on hopeless nodes after 300 iterations where shown (:300); warm: children start from the parent's multiplier.

instance	bounds	nodes	CPU s	wall
nug20	QPB only (1 core)	526,205	197	–
nug20	QPB + DNN 2–3 (1 core)	141	105	–
nug24	QPB only	≈25 M	≈5,000	≈25 min
nug24	QPB + DNN 2–3	299	796	4.4 min
nug25	QPB only	52,989,998	18,353	78 min
nug25	QPB only (`nug_deep`)	40,183,792	12,362	53 min
nug25	QPB (`nug_deep`) + DNN 2–5	717	3,279	15 min
nug25	QPB (`nug_deep`) + DNN 2–5:300	761	1,447	6.2 min
nug27	QPB (`nug_deep`) + DNN 2–5	675	4,428	20 min
nug28	QPB (`nug_deep`) + DNN 2–5	1,736	12,976	62 min
nug30	QPB (`nug_deep`) + DNN 2–5:300, then 2–6:300	45,486,332	40,555	≈3.0 h
nug30	QPB (`nug_deep`) + DNN 2–6:300	6,532,419	18,614	≈1.3 h
on a second machine, about 1.45× slower per DNN iteration:				
nug30	QPB (`nug_deep`) + DNN 2–6:300	6,532,419	28,428	≈2.1 h
nug30	QPB (`nug_deep`) + DNN 2–7:300, warm	5,817	23,618	1.7 h
on an Apple M2 Pro laptop, 8 workers, Accelerate BLAS:				
nug30	QPB (`nug_deep`) + DNN 2–7:300, warm	585,550	8,355	19.7 min

take 17% less CPU, with 5,817 nodes in all: every node at depths 2–7 that QPB does not fathom gets the DNN bound, and all of them are fathomed by depth 7. Warm starts lower the DNN time per node at depths 3 and 4 by 16% and 13%.

The same settings on the laptop solve nug30 in 19.7 minutes. Each DNN node takes about a third of the time it takes on the cloud Xeon (1.9 against 5.8 seconds at depth 3, 1.2 against 3.5 at depth 4), and eight solver processes give 7.1 CPU seconds per second of wall time. The DNN part of the tree is within 0.5% of the Xeon runs'. The larger node count comes from two of the 96 depth-7 nodes, which the Xeon runs fathomed and this run did not: the QPB search below them took 580,000 nodes but only 84 CPU seconds. Which nodes are warm-started depends on the scheduling, and Accelerate rounds differently from OpenBLAS; either can change when a node is given up as hopeless. Without the give-up rule and warm starts (`-x 2:5`), the laptop reproduces the Xeon trees of nug27 and nug28 exactly.

Table 6 compares the trees level by level. In 2000 no node was fathomed above level 4, and the tree grew by a factor of 20 to 25 per level to 2.7×10^9 nodes at level 10. As noted in [4], the largest number of nodes were at levels 10 and 11, the most time was spent at level 7, and the bulk of the computation was performed on nodes at levels 6–11; levels 2–7 took 48% of the time on 0.3% of the nodes. In 2026 the same 225 nodes appear at level 2, but the DNN bound fathoms 165 of them, then 94% of the nodes at level 3 and 99% at level 4, and none survive level 7. The 2026 tree has

5,819 nodes; the 2000 tree had more than 6,000 at level 3 alone. This run took 5,819 nodes and 24,768 CPU seconds, against 5,817 and 23,618 for the first run with the same settings: a node that one worker hands to another loses its parent's multiplier, so which nodes are warm-started depends on the scheduling.

The CPU figures are retained proof CPU: work done after the last checkpoint before a container restart was discarded and redone, and is not counted. The solves used solver commits `f8107dd` (depths 2–6) and `ff1f044` (depths 2–7) with OpenBLAS (`scipy-openblas32`) and GCC, and `2d7579c` with the Accelerate build of `7b05270` on the laptop; the fixes between them change no computation on nug data, whose products and sums are exact, and the regression instances' node counts are unchanged.

4.4. Open problems

We screened candidates among the instances QAPLIB lists as open by their certified root DNN gap (nug30: 2.9%, and tho30, solved in [2]: 4.2%): wil50 1.4%, sko49 3.1%, tho40 5.8%, tai50b 6.3%, tai35a 8.5%, tai40a 9.4%. At depth 2, 7 of 10 random nug30 nodes are fathomed, but none of 10 for tho40 and none of the sko49 nodes sampled, whose gap is still about 2%. At $n \approx 50$ an ADMM iteration costs about 1 second on four threads, 13× nug30's. A sample of 16 random depth-3 nodes of wil50 (cutoff 48,814) is worse still: the DNN bound fathoms 2; five converge 59 to 116 short of the cutoff, with primal and dual values in agreement; and nine are given up after 300 iterations, 300 to 515 short. The gap closes by about 130 per level from about 680 at the root, so fathoming would start around depth 5 or 6. With the grid's symmetry of order 4 that is about 3×10^4 nodes at depth 3, 1.4×10^6 at depth 4 and 6×10^7 at depth 5: 10^7 to 10^8 DNN nodes at several minutes each. The near misses show that at these depths the relaxation, not the solver, is the limit: a faster solver changes the cost of a node but not the depth at which fathoming starts. Nor do the triangle inequalities on Y help: added in Lagrangian form to the certified bound, they raised two nug30 depth-2 misses by 0.2 and 0.4 (of gaps 76 and 29) and the root not at all, since the DNN solution violates them by at most 0.02 (`experiments/dnn_cuts`). The open grid instances need a strictly stronger relaxation, such as level-2 RLT, as well as much larger parallel resources, as in [7].

5. Conclusion

Three advances made it possible to solve nug30 on a single small computer: a certified DNN bound, valid after any number of iterations and cheap enough to stop as soon as it fathoms; a branching rule that uses the geometry of grid instances at no cost; and a division of labor in which the strong bound is spent only where a single fathom removes a large subtree, and QPB, with its branching information and its eliminations, does the rest. Twenty-six years ago nug30 needed QPB's cheapness: 1.19×10^{10} nodes, each bounded in milliseconds, on hundreds of machines. Today the same search needs fewer than six thousand much stronger bounds, and finishes in 20 minutes on a laptop. In 2000 the computation took 6.9 years of HP9000 C3000 CPU time; on a cloud Xeon it now takes about 2.4×10^4 seconds, roughly 9,000 times less, and on the laptop about 8,400. A rough breakdown: per-node times of QPB deep in the tree, where little else

Table 6. Nodes per level, nug30 in 2000 and in 2026 (`-x 2:7:1:300:1`, a second run with the settings of the last row of Table 5). Fathom: the fraction of the level's nodes that were fathomed (in 2026 almost all by the DNN bound). Time: CPU seconds at the level; in 2000 summed over the grid's machines, on average 56% as fast as an HP9000 C3000 [4]; in 2026 on 2.8 GHz Xeon cores.

	2000 [4, Table 5.12]			2026		
Level	Nodes	Fathom	Time	Nodes	Fathom	Time
0	1	0.0000	25	1	0.0000	0
1	9	0.0000	609	9	0.0000	5
2	225	0.0000	17,612	225	0.7333	2,688
3	6,120	0.0000	398,354	1,608	0.9372	9,258
4	157,947	0.0173	2,934,065	2,679	0.9858	9,495
5	3,300,091	0.1575	19,752,635	976	0.9908	2,608
6	42,497,088	0.3952	59,055,117	225	0.9822	536
7	286,494,566	0.6135	82,612,511	96	1.0000	177
8	956,340,170	0.7428	31,244,385			
9	1,937,015,498	0.8212	41,471,872			
10	2,660,681,036	0.8540	45,069,685			
11	2,614,933,119	0.8742	32,951,942			
12	1,984,262,473	0.9091	19,435,198			
13	938,579,161	0.9206	7,602,716			
14	338,657,371	0.9265	2,287,052			
15	99,017,742	0.9295	573,502			
16	24,225,224	0.9320	118,176			
17	4,998,989	0.9337	20,572			
18	885,554	0.9355	3,044			
19	135,201	0.9367	390			
20	18,058	0.9311	44			
21	2,317	0.9094	5			
22	344	0.8605	1			
23	77	0.8312	0			
24	19	0.7368	0			
25	8	0.7500	0			
26	2	0.0000	0			
27	2	0.0000	0			
Total	11,892,208,412		3.46×10^8	5,819		24,768

has changed, suggest that a current core is about 20 times faster than a C3000 on this code; the QPB branch and bound itself is about 3 times faster (on nug25 with QPB alone, 12,400 seconds against 720,000 C3000-seconds in 2000 [4]), from fewer nodes and cheaper Frank–Wolfe iterations; and the DNN bound accounts for the remaining factor of about 150. Hardware accounts for part of the gain, but most of it comes from the bound and from where it is spent.

Several directions remain. The give-up rule and the depths of application were tuned by hand. A QPB threshold predicts the DNN outcome almost perfectly in the samples, which suggests skipping nodes the bound will miss. The restricted Peaceman–Rachford method of [9] may converge faster than ADMM, and a separate verified check of each fathoming certificate would separate fast search from rigorous verification. For the larger open grid instances speed is not enough: the DNN relaxation is too weak at the depths where their trees are wide, and a stronger relaxation is the next step.

A. Validation

A branch-and-bound proof is only as good as its bounds, so the DNN bound was checked in several independent ways.

- **Exhaustive checks.** 1,512 certified bounds on random instances and subproblems with $m \leq 8$ (several families, including linear terms and ties), each compared with the optimum found by enumeration. No bound exceeds the optimum, and no fathom is ever claimed with the cutoff at the optimum.
- **Inside the search.** The solver's brute-force test (4,536 runs on random instances with $n \leq 8$, eight parameter sets, three upper-bound modes) passes with the DNN bound applied at every depth, with and without the give-up rule.
- **Against the prototype.** In prototype mode the C++ bound reproduces the numpy prototype's certified values at 44 checkpoints to a relative $8.6 \cdot 10^{-8}$. On nug20 nodes whose subproblems were solved exactly, the bounds are below the optima and fathom the same nodes.
- **Reproducibility.** Results are bitwise identical across runs on one machine. Valgrind reports no memory errors on a debug build. Serial, resumed and parallel runs give identical node counts.
- **Independent mathematical review.** A separate AI agent, given the code and this claim but not our arguments, re-derived every step: the face basis actually used (Householder reflector and rotation) has the required range, so Lemma 1 holds for it; Proposition 2 holds for any sign of $\lambda_{\max}$; the Gilmore–Lawler evaluation counts every quadratic term once and is valid for any dual potentials, so neither the inexact inner LAPs nor the outer Hungarian solve need to be exact; the lifting matches the solver's objective convention without symmetry assumptions; a NaN or infinite multiplier cannot produce a fathom; and the search uses the DNN value only to fathom. It found no proof-breaking issue.

- **Independent code review and stress tests.** A second agent reviewed the implementation and ran 2,592 bounds on brute-forced instances with $m \leq 7$ (asymmetric and negative data, ties, scales up to 10^6), 540 warm-started children, an independent long-double check of the face transforms (error at most $2.1\,u\|Z\|_F$ against the $64\,mu\|Z\|_F$ allowed), 21,000 random assignment problems against the Hungarian solver, and determinism tests, under address, undefined-behavior and leak sanitizers. No bound exceeded an optimum and nothing crashed. Its minor findings (and the mathematical review's) were fixed: a loop that could skip the last Cholesky attempt, widened error constants, an explicit finiteness test on the certified value, and skipping the bound while there is no incumbent.

- **Adversarial tests.** A third agent tried to break the bound with extreme data: magnitudes up to 10^9 and beyond, products above 2^{53}, signed, non-integer and non-symmetric data, rank-one, constant and all-zero matrices, subproblems with one to three free facilities, and parameter fuzzing; about 15,000 bounds and 8,600 solver runs in all, with an instrumented build checking every DNN call inside the search against the exact subproblem optimum. On ordinary data no bound was invalid, and the rounding part of the margin was never needed (the unmargined value minus the semidefinite penalty stayed below the optimum by at least 1.6% of the rounding margin). It did find two ways to exceed the optimum, both needing data that is neither symmetric nor small. When neither A nor B is symmetric, an entry of L can be the difference of two large products whose rounding errors the margin, which scaled with $\sum|L|$, did not cover; the margin now uses the sum of the products' magnitudes. And the shift and linear term of a subproblem, sums of products that can cancel, were rounded in double before any bound saw them. Integer data whose products exceed 2^{53} is no longer given an integer fathoming tolerance. It also found two older bugs outside the DNN bound: a finite sentinel (10^8) in the assignment solver, which crashed or gave invalid QPB bounds when costs exceeded it (fixed), and wrong QPB results on data with negative entries in A or B (open; QAPLIB data are nonnegative). None of this affects nonnegative integer data whose objective values stay below 2^{53}, where every product and sum is exact. That includes all of QAPLIB, even the bur26 family, in which neither matrix is symmetric. On the regression instances, node counts are unchanged.

- **External review.** An independent review of an earlier draft and the code (Astra, an OpenAI Codex agent) re-derived the bound and confirmed that QPB values stay paired with their own reduced costs. It showed that accumulating the shift and linear terms in extended precision, our first repair, is not enough: with $ab = 2^{64} + 2^{34} + 3$ and $c = -(2^{64} + 2^{34})$ the extended-precision product rounds by 1 before cancelling, and both the $m = 1$ path and a reduced subproblem fathomed at a cutoff equal to the optimum. These data now get the lower bounds of item (o) in Section 2.3; both cases, and the adversarial ones above, are regression tests (`tests/dnn_test.py`, part D) that check the bound against optima computed in exact rational arithmetic.

Limitations. The certificate is a theorem in exact arithmetic (Proposition 2 and (9)). In floating point it rests on the margins of Section 2.3: standard error bounds with gen-

erous constants under the stated arithmetic model, not interval arithmetic, and not every constant is derived. The validation above is strong regression evidence, not a formal verification. For the nug instances the data are small integers, so the subproblem data are exact, and a false fathom would need a certified value above the optimum by more than the 0.05 cushion, five to six orders of magnitude above any margin observed. For non-integer data, where the tolerance is zero, validity rests on the margins alone; a verified evaluation (directed rounding or interval arithmetic) of each fathoming certificate would be the safe choice. Beyond the DNN bound, the QPB part of the search is trusted floating point, as it was in 2000.

Acknowledgments

Nathan Brixius directed the work. Jean-Pierre Goux contributed the "nug30 Laboratory" study whose ideas started it. Claude, an AI system by Anthropic, wrote the software changes, ran the experiments in Nathan Brixius's repository and drafted this report, using independent AI agents for the prototype studies, the C++ port and the reviews. Astra (an OpenAI Codex agent) reviewed an earlier draft and found the cancellation cases described in Appendix A.

References

[1] K.M. Anstreicher and N.W. Brixius, A new bound for the quadratic assignment problem based on convex quadratic programming, *Mathematical Programming* 89 (2001) 341–357.

[2] K. Anstreicher, N. Brixius, J.-P. Goux and J. Linderoth, Solving large quadratic assignment problems on computational grids, *Mathematical Programming* 91 (2002) 563–588.

[3] N.W. Brixius and K.M. Anstreicher, Solving quadratic assignment problems using convex quadratic programming relaxations, *Optimization Methods and Software* 16 (2001) 49–68.

[4] N.W. Brixius, *Solving Large-Scale Quadratic Assignment Problems*, Ph.D. thesis, University of Iowa (2000).

[5] R.E. Burkard, E. Çela, P.M. Pardalos and L.S. Pitsoulis, The quadratic assignment problem, in: D.-Z. Du and P.M. Pardalos (eds.), *Handbook of Combinatorial Optimization*, vol. 3, Kluwer (1998) 241–337.

[6] R.E. Burkard, S.E. Karisch and F. Rendl, QAPLIB – a quadratic assignment problem library, *Journal of Global Optimization* 10 (1997) 391–403.

[7] K. Fujii, N. Ito, S. Kim, M. Kojima, Y. Shinano and K.-C. Toh, Solving challenging large scale QAPs, arXiv:2101.09629 (2021).

[8] P.C. Gilmore, Optimal and suboptimal algorithms for the quadratic assignment problem, *SIAM Journal on Applied Mathematics* 10 (1962) 305–313.

[9] N. Graham, H. Hu, J. Im, X. Li and H. Wolkowicz, A restricted dual Peaceman–Rachford splitting method for a strengthened DNN relaxation for QAP, *INFORMS Journal on Computing* 34 (2022) 2125–2143.

[10] S.W. Hadley, F. Rendl and H. Wolkowicz, A new lower bound via projection for the quadratic assignment problem, *Mathematics of Operations Research* 17 (1992) 727–739.

[11] E.L. Lawler, The quadratic assignment problem, *Management Science* 9 (1963) 586–599.

[12] E.M. Loiola, N.M.M. de Abreu, P.O. Boaventura-Netto, P. Hahn and T. Querido, A survey for the quadratic assignment problem, *European Journal of Operational Research* 176 (2007) 657–690.

[13] C.E. Nugent, T.E. Vollmann and J. Ruml, An experimental comparison of techniques for the assignment of facilities to locations, *Operations Research* 16 (1968) 150–173.

[14] D.E. Oliveira, H. Wolkowicz and Y. Xu, ADMM for the SDP relaxation of the QAP, *Mathematical Programming Computation* 10 (2018) 631–658.

[15] S.M. Rump, Verification of positive definiteness, *BIT Numerical Mathematics* 46 (2006) 433–452.

[16] Q. Zhao, S.E. Karisch, F. Rendl and H. Wolkowicz, Semidefinite programming relaxations for the quadratic assignment problem, *Journal of Combinatorial Optimization* 2 (1998) 71–109.